

Integers

Integers are whole numbers, both positive and negative. For example, 456, 12, 0, -1 and -3965 are all integers, $3\frac{1}{4}$, π , -2.995, 0.888... and $\sqrt{2}$ are not integers, rather they are real numbers. Integers form a subset of the real numbers. Zero is a special integer; it is neither positive nor negative. We have already seen how positive integers and zero can be represented using binary on computers. How do we represent negative integers using only zeros and ones? We can't use a '-' sign directly as computers can only work with zeros and ones. A number of techniques have been developed. The aim of each technique is to provide a system that allows accurate and efficient calculations to take place. In this section, we examine two integer representation techniques:

- Sign and modulus.
- Two's complement (and one's complement).

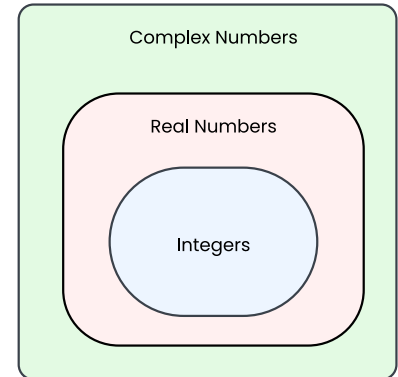


Fig 1.3.13

Sign and modulus

The sign and modulus method is perhaps the most obvious solution. We just add an extra bit to the front of each number to represent the sign of the number. A zero is used for positive numbers and a one for negative numbers. The modulus is the magnitude of the number or the absolute value of the number. Mathematically the absolute value of a number is the distance from the number to zero.

If we are using 7 bits for the modulus then the number 12 would be represented as 00001100 and -12 as 10001100. Using this system, zero can be represented in two ways; either 10000000 or 00000000.

Using the sign and modulus method, the sign of the number is not an integral part of the number. This means that calculations must take account of the sign bit separately to the actual calculation. This is essentially what we do when adding or subtracting negative numbers by hand. For example, when adding say 12 and -7 we know that adding a negative is the same as subtracting, so we perform the calculation $12 - 7$ and get the result 5. Computers find subtraction unnecessary and hence the sign and modulus method is not generally used on modern computers.

CLASS DISCUSSION

Discuss: Computer Sign and Modulus

The sign and modulus technique is difficult to implement on a computer because computers are not real smart. They just like to add things, mind you they can add at lightning speed, subtraction they find difficult and unnecessary. Can you think of any integer subtractions that cannot be rewritten as additions? Is subtraction really necessary? Discuss.

Two's complement (and one's complement)

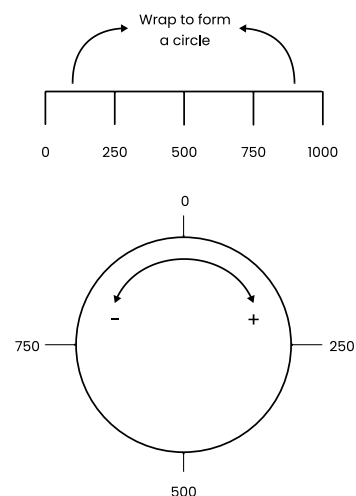
Working out a method of representing integers where the sign of the number is included as an integral part of the representation is solved using complements. Let us discuss the thinking behind this method using the decimal system before we implement it in binary.

In primary school, you most likely used a number line, in Year 7 this number line was extended to include negative numbers. Mathematically the number line extends infinitely in both positive and negative directions. For a moment, imagine we dispense with the negative or left hand side of the number line and that our positive numbers are finite, say with a maximum value of 1000. Imagine we join the ends of this finite number line to form a circle (see Fig 1.3.14).

Now if we move in a positive or clockwise direction our integers increase in value. If we move in a negative or anti-clockwise direction our integers decrease in value, with one important exception, zero. Moving anti-clockwise from zero we would like to decrease into the negative numbers, -1, -2, -3 and so on, instead we reach 999, 998, 997... These numbers are complements of their positive equivalents, in our example the ten's complement of 1 is 999, of 2 is 998, of 3 is 997 and so on. Each of these pairs of numbers is the same distance from zero on our circular number line.

We cannot have 999 representing both -1 and 999 so we need to develop some rules. Let us make sure there are an equal number of positive and negative integers. Positive's run from 0 to 499 and negatives from 999 down to 500.

Now let us test our results using normal addition. For example $-3 + -5$, using our ten's complement system this becomes $997 + 995$ giving us a result of 1992. If we discard the leading 1, we get the correct answer 992, which using our system is equivalent to -8. Let us try $-6 + 19$, this becomes $994 + 19$ resulting in 1013. Discarding the leading 1 gives us the correct result 13.



*Fig 1.3.14
Wrapping a finite
positive number line to
form a circle*

CLASS DISCUSSION

Activity: Number System

Perform the following additions using our number system developed above: $18 + -25$, $-100 + 109$, $-5 + 5$ and $-15 + -39$. Make sure you understand this system thoroughly before moving onto the next section.

Discussion: Signs in Number System

Remember the aim was to develop a system where no sign is required for negative numbers. How has this been achieved? Does the above number system work with subtraction, multiplication and division?

We now examine this system as it is applied to binary numbers. Generally, integers are stored within computers using 16, 32 or 64 bits. For our discussion, we will use 8 bits to help simplify the mathematics. With 8 bits, we can represent numbers from 00000000 up to 11111111. If we form a circular number line using these numbers, we see that 11111111 should be used to represent -1. Fig 1.3.15 shows the significant part of this circular number line on either side of zero.

Let us examine pairs of positive and negative numbers using this system. Fig 1.3.16 shows the first sixteen integers along with their two's complement. For example, the two's complement of 00000001 is 11111111.

A number of patterns emerge. Firstly, each positive number commences with a 0 and each negative with a 1. In fact, it turns out that this is always the case regardless of the number of bits used. Secondly, exchanging 1s for 0s and 0s for 1s almost results in the correct two's complement, in fact the result is out by 1. For example, 7 is 00000111, swapping 1s and 0s gives 11111000, which has a value of -8. If we add 1 to this result, we get the correct answer 11111001.

This process of swapping 1s and 0s results in the one's complement of the number. Why is it called the one's complement? Because the swapping of 1s and 0s can be achieved by subtracting the number from a string of 1s. For example, 11111111 - 00110111 results in 11001000, hence the name one's complement.

It is possible to perform arithmetic using ones complement numbers, however the two's complement system proves to operate more efficiently. Further analysis of our two's complement table in Fig 1.3.16 reveals another simpler method of calculating two's complements. Begin reading the original binary number from the right hand side moving one digit at a time to the left. As soon as a one is encountered commence swapping 1s and 0s, just don't change the first one. For example, if we wish to find the two's complement of 12, which is 00001100 in binary. Scanning from the right hand side, we encounter a 1 in the third position from the right. Leave this 1 and then swap all the 1s and 0s to its left; the result is 11110100, which is correct!

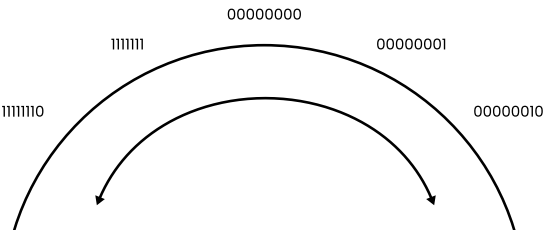


Fig 1.3.15
Section of an 8 bit
circular number line

Positive		Negative	
Dec	Bin	Dec	Bin
1	00000001	-1	11111111
2	00000010	-2	11111110
3	00000011	-3	11111101
4	00000100	-4	11111100
5	00000101	-5	11111011
6	00000110	-6	11111010
7	00000111	-7	11111001
8	00001000	-8	11111000
9	00001001	-9	11111011
10	00001010	-10	11111010
11	00001011	-11	11111001
12	00001100	-12	11111000
13	00001101	-13	11111011
14	00001110	-14	11111010
15	00001111	-15	11111001
16	00010000	-16	11111000

Fig 1.3.16
Two's complements for
the first sixteen integers

CLASS ACTIVITY

Activity: 8-bit integers

Extend the table in Fig 10.15 to include all possible 8-bit integers that can be represented. What is the range of integers represented using 8 bits?

Activity: 8-bit two's complement numbers

Write down ten 8-bit two's complement numbers, 5 positive and 5 negative. Convert each number to its two's complement. Confirm that the system works to convert negatives to positives as well as positives to negatives.

We mentioned above that all positive two's complement numbers commence with a zero and all negatives with a one. This information is particularly useful if we wish to convert a two's complement number into decimal.

Using our 8-bit two's complement system -12 is represented as 11110100. Each binary digit has a place value, commencing from the right hand side with $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 16$, $2^5 = 32$, $2^6 = 64$ and the last or left hand bit has a place value of $-2^7 = -128$. The bit on the left hand side is often called the 'highest order bit' or the 'most significant bit'. We can write 11110100 as $-128 + 64 + 32 + 16 + 0 + 4 + 0 + 0$ which equals -12. If we were performing many of these conversions, it is convenient to draw up a table such as the one in Fig 1.3.17. Add up all the positive values first and, if necessary, subtract 128 at the end.

-128	64	32	16	8	4	2	1	Decimal
1	1	1	1	0	1	0	0	-12
0	1	0	1	1	0	1	0	90
1	0	0	0	0	1	0	1	-123
0	0	0	0	1	1	0	1	13

64

32

16

4

116

- 128

- 12

64

16

8

2

90

- 128

4

1

5

- 128

- 123

8

4

1

13

Working

Fig 1.3.17

CLASS ACTIVITY

Activity: Two's complement to decimal

Draw up a table, like the one in Fig 1.3.17. Randomly fill in the table with ones and zeros. Convert each number to its decimal equivalent.

Activity: Range of integers

Most computers use 16, 32 and/or 64-bit two's complement to represent integers. What is the range of integers that can be represented using each of these three representations?

Activity: Python Range of integers

Python is able to represent an arbitrarily large range of integers. Research how this is achieved.